

# CSS Animations — Concepts and Examples

---

## Introduction to CSS Animation

---

CSS animations allow us to change the appearance or position of an HTML element over a period of time.

With CSS animations, we can make elements:

- Change colors
- Move from one position to another
- Change size
- Rotate
- Change opacity
- Move along different paths
- Perform an animation repeatedly
- Start after a delay
- Run faster or slower
- Run forward or backward

CSS animations are created mainly using two things:

1. `@keyframes` — Defines **what changes during the animation**
2. `animation-*` properties — Defines **how the animation behaves**

## Basic structure

```
@keyframes animationName {  
  from {  
    /* starting state */  
  }  
  
  to {  
    /* ending state */  
  }  
}
```

Then the animation is applied to an HTML element:

```
.box {  
  animation-name: animationName;  
  animation-duration: 3s;
```

```
}
```

---

# What is @keyframes ?

---

@keyframes is an important CSS rule used to define the different stages of an animation.

Think of keyframes as instructions that tell the browser:

"At this point in the animation, the element should look like this."

For example:

```
.box {  
  animation-name: colorChange;  
  animation-duration: 3s;  
}
```

```
@keyframes colorChange {  
  from {  
    background-color: red;  
  }  
  
  to {  
    background-color: yellow;  
  }  
}
```

Here:

- colorChange is the animation name.
- from represents the starting state.
- to represents the ending state.
- background-color changes during the animation.

---

## from and to

---

The simplest form of @keyframes uses:

```
from
```

and

to

## Example

```
@keyframes changeColor {  
  from {  
    background-color: red;  
  }  
  
  to {  
    background-color: yellow;  
  }  
}
```

The browser automatically calculates the intermediate states.

Conceptually:

```
Red  
|  
| gradually changes  
|  
|  
Yellow
```

The developer does not need to manually specify every intermediate color.

---

## Percentage-Based Keyframes

Instead of `from` and `to`, we can use percentages.

The animation timeline starts at:

0%

and ends at:

100%

Example:

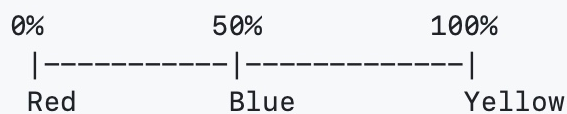
---

```
@keyframes changeColor {
  0% {
    background-color: red;
  }

  50% {
    background-color: blue;
  }

  100% {
    background-color: yellow;
  }
}
```

The animation now has three stages:



Percentage-based keyframes are useful when an animation has multiple stages.

---

## Why Do We Need Keyframes?

---

Suppose we want an element to perform this sequence:

```
Start
↓
Change color
↓
Move
↓
Rotate
↓
Change size
↓
End
```

A simple `from` and `to` animation may not be enough.

Keyframes allow us to describe multiple stages:

```
@keyframes example {
  0% {
    background-color: red;
  }
```

```
25% {
  background-color: blue;
}

50% {
  transform: translateX(100px);
}

75% {
  transform: rotate(180deg);
}

100% {
  background-color: yellow;
}
}
```

---

## animation-name

---

The `animation-name` property specifies which `@keyframes` animation should be used.

Example:

```
@keyframes changeColor {
  from {
    background-color: red;
  }

  to {
    background-color: yellow;
  }
}

.box {
  animation-name: changeColor;
}
```

Here:

```
animation-name: changeColor;
```

connects the element with:

```
@keyframes changeColor
```

The names must match

The names must match.

---

## animation-duration

---

The `animation-duration` property specifies how long one complete animation cycle should take.

Example:

```
.box {  
    animation-duration: 5s;  
}
```

Here:

5s = 5 seconds

Another example:

```
.box {  
    animation-duration: 2s;  
}
```

This makes one animation cycle take 2 seconds.

### Common units

```
animation-duration: 2s;
```

```
animation-duration: 500ms;
```

1s is equal to 1000ms .

Therefore:

```
1s    = 1000ms  
2s    = 2000ms  
0.5s  = 500ms
```

# Animation Delay

---

The `animation-delay` property specifies how long the browser should wait before starting the animation.

Example:

```
.box {  
  animation-name: changeColor;  
  animation-duration: 3s;  
  animation-delay: 2s;  
}
```

The browser waits for 2 seconds and then starts the animation.

Conceptually:

```
Page loads  
  |  
  | 2 seconds  
  ↓  
Animation starts  
  |  
  | 3 seconds  
  ↓  
Animation finishes
```

The delay affects the **start time**, not the speed of the animation.

---

## animation-iteration-count

---

The `animation-iteration-count` property specifies how many times an animation should run.

Example:

```
.box {  
  animation-name: changeColor;  
  animation-duration: 3s;  
  animation-iteration-count: 3;  
}
```

The animation runs three times.

---

Cycle 1  
Cycle 2  
Cycle 3

---

## Infinite Animation

---

The special value:

`infinite`

makes the animation repeat continuously.

Example:

```
.box {  
  animation-name: changeColor;  
  animation-duration: 3s;  
  animation-iteration-count: infinite;  
}
```

Conceptually:

```
Animation  
  ↓  
Cycle 1  
  ↓  
Cycle 2  
  ↓  
Cycle 3  
  ↓  
Cycle 4  
  ↓  
...  
  ↓  
Continues
```

The animation does not stop automatically.

---

## Animation Direction

---

The `animation-direction` property determines the direction in which the animation should run.

Important values include:

```
normal  
reverse  
alternate  
alternate-reverse
```

---

## normal

---

This is the default direction.

The animation runs from the first keyframe to the last keyframe.

```
.box {  
  animation-direction: normal;  
}
```

Conceptually:

Start → End

---

## reverse

---

With `reverse`, the animation runs in the opposite direction.

```
.box {  
  animation-direction: reverse;  
}
```

Conceptually:

End → Start

The keyframes themselves do not change.

The browser simply plays them in reverse order.

---

## alternate

---

`alternate` changes direction on every iteration.

Example:

```
.box {  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}
```

Conceptually:

```
Start → End  
End   → Start  
Start → End  
End   → Start  
...
```

This is useful for effects where an element should move back and forth.

---

## alternate-reverse

This is similar to `alternate`, but the first iteration begins in the reverse direction.

```
.box {  
  animation-direction: alternate-reverse;  
}
```

Conceptually:

```
End → Start  
Start → End  
End → Start  
...
```

---

## Animation Timing Function

The `animation-timing-function` property controls **how the animation progresses between keyframes**.

It affects the speed pattern of the animation.

It does not necessarily change the total animation duration.

For example, two animations may both take 5 seconds, but one may move smoothly while another may start slowly and then become faster.

---

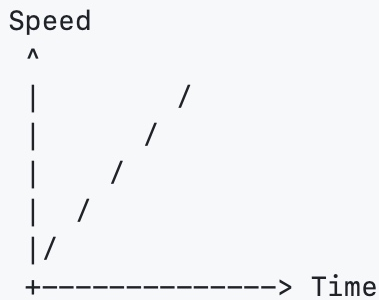
## linear

---

With `linear`, the animation progresses at a constant rate.

```
.box {  
  animation-timing-function: linear;  
}
```

Conceptually:



The element moves at approximately the same rate throughout the animation.

---

## ease

---

`ease` is the default timing function.

```
.box {  
  animation-timing-function: ease;  
}
```

It generally starts relatively slowly, speeds up, and slows down toward the end.

Conceptually:

Slow → Fast → Slow

---

# ease-in

---

`ease-in` starts slowly and then becomes faster.

```
.box {  
  animation-timing-function: ease-in;  
}
```

Conceptually:

Slow → Fast

This can be useful when an object appears to gradually accelerate.

---

# ease-out

---

`ease-out` starts faster and slows down toward the end.

```
.box {  
  animation-timing-function: ease-out;  
}
```

Conceptually:

Fast → Slow

---

# ease-in-out

---

`ease-in-out` starts slowly, becomes faster, and then slows down again.

```
.box {  
  animation-timing-function: ease-in-out;  
}
```

Conceptually:

Slow → Fast → Slow

---

# Comparing Timing Functions

---

| Timing Function | Behavior           |
|-----------------|--------------------|
| linear          | Constant speed     |
| ease            | Slow → Fast → Slow |
| ease-in         | Slow → Fast        |
| ease-out        | Fast → Slow        |
| ease-in-out     | Slow → Fast → Slow |

---

# Changing the Background Color

---

CSS animations can change the `background-color` property.

Example:

```
@keyframes colorAnimation {  
  from {  
    background-color: red;  
  }  
  
  to {  
    background-color: yellow;  
  }  
}
```

Apply it to an element:

```
.box {  
  width: 100px;  
  height: 100px;  
  
  animation-name: colorAnimation;  
  animation-duration: 3s;  
}
```

The browser gradually changes the background color during the animation.

---

# Moving an Element with `transform`

CSS animations can also change the position of an element using `transform`.

One commonly used function is:

```
translate()
```

There are also:

```
translateX()  
translateY()
```

---

## `translateX()`

`translateX()` moves an element horizontally.

Example:

```
transform: translateX(200px);
```

Positive values generally move the element toward the right.

Conceptually:

Original position



[BOX]



[BOX]

Example animation:

```
@keyframes moveHorizontal {  
  from {  
    transform: translateX(0);  
  }  
}
```

```
    to {
        transform: translateX(200px);
    }
}
```

---

## translateY()

---

`translateY()` moves an element vertically.

Example:

```
transform: translateY(200px);
```

Positive values generally move the element downward.

Conceptually:

```
[BOX]
|
|
|
↓
[BOX]
```

Example:

```
@keyframes moveVertical {
    from {
        transform: translateY(0);
    }

    to {
        transform: translateY(200px);
    }
}
```

---

## Combining Horizontal and Vertical Movement

---

We can use both X and Y values.

Example:

---

```
transform: translate(200px, 100px);
```

Here:

200px → horizontal movement  
100px → vertical movement

Example:

```
@keyframes move {  
  from {  
    transform: translate(0, 0);  
  }  
  
  to {  
    transform: translate(200px, 100px);  
  }  
}
```

This allows an element to move diagonally.

---

## Moving Through Multiple Positions

---

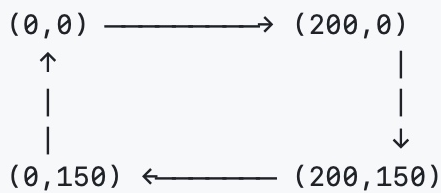
Keyframes become especially useful when an element needs to pass through multiple positions.

Example:

```
@keyframes path {  
  0% {  
    transform: translate(0, 0);  
  }  
  
  25% {  
    transform: translate(200px, 0);  
  }  
  
  50% {  
    transform: translate(200px, 150px);  
  }  
  
  75% {  
    transform: translate(0, 150px);  
  }  
  
  100% {
```

```
    transform: translate(0, 0);  
  }  
}
```

Conceptually:



This demonstrates how multiple keyframes can create a path.

---

## The Relationship Between **transform** and Animation

---

**transform** describes what transformation should happen.

**animation** describes how that transformation changes over time.

For example:

```
transform: translateX(200px);
```

means:

Move the element horizontally by 200 pixels.

But:

```
@keyframes move {  
  from {  
    transform: translateX(0);  
  }  
  
  to {  
    transform: translateX(200px);  
  }  
}
```

means:

Gradually change the element from one position to another during the animation.

---

# Other Useful Transform Functions

---

CSS provides several transformation functions.

## Rotate

```
transform: rotate(45deg);
```

Rotates an element.

## Scale

```
transform: scale(1.5);
```

Makes an element larger.

## Translate

```
transform: translate(100px, 50px);
```

Moves an element.

## Combining transformations

```
transform: translateX(100px) rotate(45deg);
```

An element can have more than one transformation.

---

# Animation Fill Mode

---

The `animation-fill-mode` property determines what styles should be applied before and/or after an animation.

Important values include:

```
none  
forwards  
backwards
```

backwards  
both

---

## forwards

---

With `forwards`, the element keeps the styles from the final keyframe after the animation finishes.

Example:

```
.box {  
  animation-fill-mode: forwards;  
}
```

Without an appropriate fill mode, the element may return to its original CSS state after the animation ends.

---

## backwards

---

`backwards` applies the styles from the first relevant keyframe during the animation delay.

```
.box {  
  animation-fill-mode: forwards;  
}
```

---

# Animation Shorthand Property

---

Instead of writing several animation properties separately:

```
.box {  
  animation-name: example;  
  animation-duration: 5s;  
  animation-delay: 2s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
  animation-timing-function: linear;  
}
```

CSS provides the shorthand property:

CSS provides the shorthand property.

```
animation
```

Example:

```
.box {  
    animation: example 5s linear 2s infinite alternate;  
}
```

The shorthand can make CSS shorter.

However, while learning, it is often easier to first understand the individual properties.

---

## Complete Basic Animation Example

Here is a simple independent example demonstrating several animation concepts.

### HTML

---

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>CSS Animation</title>  
  
    <style>  
        .box {  
            width: 100px;  
            height: 100px;  
            background-color: red;  
  
            animation-name: example;  
            animation-duration: 4s;  
            animation-delay: 1s;  
            animation-iteration-count: 2;  
            animation-direction: normal;  
            animation-timing-function: ease-in-out;  
        }  
  
        @keyframes example {  
            0% {  
                background-color: red;  
                transform: translateX(0);  
            }  
  
            50% {  
                background-color: blue;
```

```
        transform: translateX(200px);
    }

    100% {
        background-color: yellow;
        transform: translateX(400px);
    }
}
</style>
</head>

<body>

    <div class="box"></div>

</body>
</html>
```

This example demonstrates:

```
@keyframes
animation-name
animation-duration
animation-delay
animation-iteration-count
animation-direction
animation-timing-function
background-color
transform
translateX()
percentage keyframes
```

.

---

## Quick Reference

---

| Property                  | Purpose                                  |
|---------------------------|------------------------------------------|
| @keyframes                | Defines animation stages                 |
| animation-name            | Selects the keyframe animation           |
| animation-duration        | Defines how long one cycle takes         |
| animation-delay           | Defines how long to wait before starting |
| animation-iteration-count | Defines how many times animation runs    |

|                                        |                                           |
|----------------------------------------|-------------------------------------------|
| <code>animation-direction</code>       | Defines animation direction               |
| <code>animation-timing-function</code> | Controls speed progression                |
| <code>animation-fill-mode</code>       | Controls styles before/after animation    |
| <code>animation-play-state</code>      | Pauses or resumes animation               |
| <code>transform</code>                 | Applies movement, rotation, scaling, etc. |
| <code>translateX()</code>              | Moves horizontally                        |
| <code>translateY()</code>              | Moves vertically                          |
| <code>translate()</code>               | Moves horizontally and vertically         |
| <code>rotate()</code>                  | Rotates an element                        |
| <code>scale()</code>                   | Changes the size of an element            |